

Trabajo Final Integrador

Asignatura: *Taller de Control de Calidad de Software*

Comentarios: En rojo esta lo q suena muy chatgepete hay que hacerlo sonar más humano
El punto de partida con el que vamos a empezar a resolver este trabajo es asumir que la casuística es el reflejo de lo que el sistema hace y de cómo se espera que se vea, y eso lo ve toda la organización de nuestro trabajo de testing. Por eso conviene estructurar los casos en suites organizadas por funcionalidad, en este caso con Orange HRM (Employee List, Add Employee, Edit Employee – Report to, My Info – Qualifications, Admin – Add, Widget Clima, etc.), porque la funcionalidad es la unidad que podemos mantener: cuando una historia cambia, sabemos qué casos tocar y cuáles no. Dentro de cada funcionalidad separamos dos planos que responden a preguntas distintas. Las pruebas funcionales responden a qué hace el sistema. Y las diseñamos con técnicas de caja negra, describiendo flujo, datos de entrada, estado inicial, resultado esperado y estado final, con su **oráculo** correspondiente.

Las pruebas de look&feel responden a cómo debe verse el programa y verifican que se respeten los lineamientos de diseño: colores, tipografía, diagramación, comportamiento responsive, resolución soportada, modo oscuro, por ejemplo, contra las maquetas provistas por el equipo de UI/UX.

Las mantenemos separadas porque tienen ciclo de vida, criterios de aprobación y frecuencia de ejecución diferentes, lo funcional cambia con la lógica de negocio y se puede automatizar, mientras que el look&feel cambia con el rediseño visual, depende del navegador, sistema operativo o dispositivo y se valida de forma visual. **Un recurso clave acá es trabajar con casos abstractos, que luego instanciamos en casos específicos con datos concretos; así un mismo caso funcional abstracto de Login válido puede ejecutarse con distintos datos y, en muchos casos, reutilizarse entre la versión web y la mobile.**

Esa reutilización tiene un límite importante: como la mobile es web acotada, muchos casos funcionales sirven igual, pero los casos de look&feel casi nunca se trasladan tal cual, porque el aspecto varía entre dispositivos, sistemas operativos y tamaños de pantalla, y hay que diseñar casos alternativos específicos para mobile. Toda esta casuística la gestionamos idealmente en una herramienta de test management que permita versionar las suites, marcar cuáles son de smoke y cuáles de regresión, y registrar las corridas (test runs).

Esa misma lógica de funcionalidad viva nos permite encarar el problema de probar el frontend cuando el backend todavía no está listo, que es habitual en un equipo donde front y back avanzan en paralelo dentro del mismo sprint. La estrategia es desacoplar la prueba del front de la disponibilidad del back mediante mocks / virtualización de servicios: simulamos la respuesta de cada microservicio (con herramientas tipo los mock services que vimos, o stubs) devolviendo respuestas controladas según el contrato acordado.

Para que esto funcione, lo primero que debemos fijar es el contrato de la API (endpoints, formato de request/response, códigos de estado, estructura de los JSON), idealmente antes de hacer código, de manera que back y front desarrollen contra el mismo acuerdo. Con el mock respondiendo esos contratos, el front puede probarse tempranamente en su comportamiento, navegación, validaciones de pantalla, manejo de estados de carga y de error, sobre todo, en look&feel, que no depende del backend en absoluto.

Esto es lo que se refirieron en las ultimas clases como shift-left: probar lo antes posible, encontrar defectos cuando corregirlos es barato y no esperar a que esté todo integrado y sea un problema modificarlo en producción. El criterio para volver a "lo real" es que, una vez

disponible el servicio verdadero, esos mismos casos los reejecutamos contra el backend real para validar la integración efectiva, de manera que el mock sirvió para adelantar la prueba pero no la reemplaza.

Del lado del backend, organizamos la cobertura de cada servicio para que pueda correr automáticamente en cada despliegue con cambios, que es donde la automatización rinde.

Por cada microservicio definimos un conjunto de casos que cubra el contrato completo: para cada endpoint, el camino feliz y los caminos no felices o alternativos. Si el servicio maneja entidades, buscamos cubrir el ciclo completo de cada entidad relevante (crear, leer, actualizar, borrar) e incluso ejecutar una lectura después de cada actualización para confirmar que el dato se refleja bien donde se consulta.

Estos casos los escribimos y guardamos en una herramienta que permita repetirlos (Postman con sus colecciones, o frameworks como RestAssured/Karate para la versión automatizada), y los integramos al pipeline de CI/CD para que se disparen ante cada commit/merge y cada despliegue. La práctica que ata todo es la suite de regresión automatizada por servicio más una suite de smoke —subconjunto crítico que valida que los caminos felices y los no felices críticos siguen funcionando— que corre primero y rápido para abortar despliegues rotos.

Conceptualmente esto sigue la idea de pirámide de niveles: muchas pruebas de componente/unitarias pruebas de integración entre componentes y entre servicios (que en microservicios es donde más riesgo hay, por las interfaces), y pruebas de sistema end-to-end. Como buena práctica complementaria conviene sumar contract testing (validar que el proveedor sigue cumpliendo el contrato que el front consume, exactamente lo que pide la Actividad 3 con la API del Clima), validación de entradas, manejo correcto de códigos HTTP y, en el pipeline, análisis estático y chequeos de seguridad automatizados.

Ahora bien, que todo eso funcione en un ambiente controlado no garantiza que sobreviva a producción con picos de tráfico concentrados en pocas horas, como el caso de los empleados fichando ingreso o salida por turnos.

Ahí aparecen problemas que son típicamente no funcionales: degradación o caída de los tiempos de respuesta cuando miles de usuarios usan los mismos endpoints concurrentemente; cuellos de botella en la base de datos; agotamiento de pools de conexiones, hilos o memoria;; saturación de algún microservicio que arrastra al resto por el acoplamiento entre servicios; errores que solo aparecen en concurrencia (condiciones de carrera, doble registro de una misma marcación); y rate limiting o throttling mal calibrados que terminan rechazando solicitudes legítimas en el peor momento. Son, en definitiva, problemas de rendimiento, escalabilidad y estabilidad que no se ven con casos funcionales y que, si se descubren en vivo, ya son incidentes.

La forma de anticiparlos antes de que sean problemas reales es someter al sistema a pruebas de performance en un entorno lo más parecido a producción posible, con un volumen de datos importantes, y atacar el problema de forma temprana y no recién al final.

El menú de actividades que organizaríamos como equipo de testing especializado en performance incluye: pruebas de carga simulando el número de usuarios y acciones del

pico real (el fichaje a la mañana o tarde) para ver cómo responde ante carga típica; pruebas de estrés subiendo la carga hasta el punto de saturación para conocer el techo de la infraestructura actual y cómo se recupera tras el pico; pruebas de resistencia/estabilidad con carga constante durante un período prolongado para cazar fugas de memoria y degradaciones lentas; pruebas de volumen aumentando gradualmente el volumen de datos para ver cómo responde la base a medida que crece el histórico de fichadas; y pruebas de configuración para medir el impacto de distintos ajustes de servidor/red/caché.

Todo esto con herramientas como JMeter, Gatling, Locust, Artillery o BlazeMeter para generar la carga, y Grafana para monitorear y analizar métricas (tiempos de respuesta, tasas de error, uso de recursos) e identificar el cuello de botella, que debe interpretarse de forma multidisciplinaria entre testers, desarrolladores y quien administre infra.

En cuanto a la frecuencia, proponemos escalonarla: pruebas de performance acotadas (sobre los endpoints críticos como la fichada) en cada sprint o ante cada cambio que toque esos servicios, integradas al pipeline para tener feedback temprano y barato; una prueba de carga de regresión completa antes de cada release importante a producción; pruebas de resistencia/soak con menor periodicidad (por ejemplo previo a cambios mayores de arquitectura o base) porque son largas y costosas; y monitoreo continuo en producción que retroalimenta los escenarios de prueba con datos reales de los picos. El ciclo es siempre el mismo: ejecutar, validar que la prueba fue válida, analizar, detectar la oportunidad de mejora (caché, índices en base, tuning de infra, revisión de la arquitectura de microservicios), implementarla y reejecutar la misma prueba para verificar empíricamente la mejora antes de subir la carga, porque no siempre el cambio que creemos que mejora, mejora.

Para que todo este volumen de pruebas funcionales, look&feel, de API, de regresión y de performance entre dentro de cada sprint, organizamos el equipo de testing apoyándonos en los principios del testing ágil: el testing no es una fase final sino una actividad permanente y continua, la realiza todo el equipo y conduce el desarrollo.

Con los tres testers fullstack disponibles, el reparto natural es que el tester con perfil de líder coordine la estrategia, los criterios de aceptación, la priorización del riesgo y el reporte de estado ("salud" de los desarrollos); que el tester de performance se ocupe del plano no funcional descrito arriba; y que entre los tres nos distribuyamos las funcionalidades del sprint para diseñar y ejecutar casos funcionales y de look&feel, automatizando lo que vale la pena (regresión, smoke, API) y dejando para lo manual la exploración y el look&feel que requiere ojo humano. Como el testing arranca al inicio del sprint, participamos desde el refinamiento revisando historias y criterios de aceptación (otra forma de shift-left que detecta ambigüedades antes de codificar), diseñamos casos en paralelo al desarrollo, ejecutamos tempranamente contra mocks y luego contra lo real, y corremos la regresión/smoke de forma continua mediante CI/CD. Conviene apoyarse en checklists y documentar lo imprescindible para no frenarse, dado que en Ágil se prioriza menos documentación y más feedback. La regla práctica es que la suite de smoke se ejecute en cada iteración y crezca en cada iteración, sumando los escenarios del incremento nuevo, de modo que la red de seguridad se va ampliando sprint a sprint sin volverse impagable. Si en algún sprint el volumen excede nuestra capacidad (por ejemplo un sprint cargado de performance y de regresión simultánea), es válido y recomendable proponer sumar un perfil puntual, tal como habilita el enunciado.

Finalmente, esa misma maquinaria es la que reaprovecharíamos para incorporar recruitment en tres sprints adicionales una vez en producción, sin necesidad de hacer un plan de pruebas nuevo: se trata de aplicar el plan existente a un feature nuevo. Lo primero es descomponer recruitment en incrementos verticales entregables por sprint, agrupados por funcionalidad y priorizados por valor y riesgo de negocio, por ejemplo, un primer sprint con la base (publicación/gestión de vacantes y su backend), un segundo con la postulación de candidatos y carga de CV, y un tercero con el flujo de evaluación/seguimiento y la integración con los módulos ya existentes (PIM, Admin)— de modo que cada sprint deje algo demostrable y probado. En cuanto a recursos, el equipo base alcanza, manteniendo el reparto de los tres testers; donde conviene reforzar es en el analista funcional para definir bien los criterios de aceptación del nuevo dominio y, eventualmente, sumar un perfil puntual si algún sprint concentra mucha integración o validación de seguridad (Recruitment maneja datos personales de candidatos, lo que eleva el peso de las pruebas de seguridad y de manejo de datos). El enfoque de pruebas replica lo anterior: fijamos contratos de los nuevos servicios para habilitar pruebas tempranas del front con mocks; diseñamos casuística funcional y de look&feel por cada nueva pantalla, reutilizando casos abstractos donde aplique y creando alternativos para mobile; armamos las colecciones de API con camino feliz y no feliz para cada endpoint nuevo, integrándolas al pipeline; y —punto crítico— ampliamos la regresión y el smoke en cada sprint para garantizar que la incorporación de Recruitment no rompe la funcionalidad de Gestión de Personas que ya está viva en producción, que es precisamente el riesgo principal de seguir agregando features sobre un sistema en uso. Sobre el plano no funcional, conviene evaluar desde el inicio si Recruitment introduce nuevos picos de carga (por ejemplo, aperturas masivas de búsquedas) e incluir, donde corresponda, las mismas pruebas de carga y estrés sobre sus endpoints antes de liberarlo.